

Interview Guide

Recovering Software Architecture Intent from Historical Work Items using GenAI

Interviewer:	Dominik Storck
Duration:	~45 minutes
Target Audience:	Software Practitioners at [Industry Partner]
Material:	Project specific, automatically generated System Context and Container Diagrams, embedded in accompanying pptx slides

Part 1: Introduction & Warm-up (8 Minutes)

Goal: Create comfortable, constructive atmosphere, ensure confidentiality

- Welcome
 - Appreciation of Participation
 - Research Topic: how LLMs can generate architecture diagrams from work items (requirements).
- Confidentiality
 - Responses will be anonymized and used solely for this academic thesis.
 - Consent to record this session and potentially publish anonymized transcripts
- Interview Content:
 - Briefly introduce procedure and content of the interview:
 - Welcome
 - Interview Context
 - Evaluation in three steps: 2 diagrams, in each step, looking at both)
- Warm-up, verification of technical Expertise and familiarity with evaluation project:
 - Could you briefly describe your current role at [Industry Partner]?
 - How do you currently document or visualize architecture in your projects?

- What is your involvement/familiarity with the project?

Part 2: Context Setting (5 Minutes)

Goal: Introduce my Research and the C4 Model.

- My Research: develop and evaluate reproducible and deterministic workflow that leverages LLMs to semi automatically generate Architecture Diagrams from Requirements (Azure DevOps Work Items) to support software teams.
- Are you familiar with the C4 model notation? In case no: C4 Model Pitch:

"The C4 Model by Simon Brown is a hierarchical framework for visualizing software architecture, designed to solve the problem of ambiguity in technical documentation. It utilizes a static structural decomposition method analogous to the zoom levels in cartography, allowing architects to model systems at four distinct levels of abstraction:

1. Context: The highest level, defining system boundaries and interactions with external actors.
2. Containers: The architectural building blocks (e.g., applications, data stores, microservices).
3. Components: The logical grouping of functionality within those containers.
4. Code: The granular implementation details (classes and interfaces).

By strictly defining these layers, the C4 model provides a standardized notation for communicating architectural intent across varying levels of technical granularity."

- My Research focuses on first two layers: System Context and Container view
- My Tool: The diagrams you are about to see were generated directly from the project's Azure DevOps Work Items using my prototype tool.

Part 3: Focus Area 1 - Understandability & Readability (10 Minutes)

Goal: Assess RQ1 Understandability.

A. System Context Diagram (5 Minutes)

- Guiding Question:
 1. At first glance and without analyzing the correctness yet, what is the most dominant feature that grabs your attention?

2. Based strictly on this visual, how would you describe the responsibilities of the System Under Design to a new colleague?
- Follow-up Questions:
 1. Which specific visual elements or relationships were immediately clear to you, and which required a second look?
 2. Is there any information or element here that feels 'noisy' or visually distracting from the core systems?

B. Container Diagram (5 Minutes)

- Guiding Question:
 1. Zooming in to the Container level: How easy is it to distinguish the internal containers from external systems here?
 2. How intuitive is it for you to identify the major technical building blocks and their responsibilities solely from this visual representation?
- Follow-up Questions:
 3. How would you interpret the relationship between Container A and B based on this label?
 4. If you handed this diagram to a new developer during onboarding, what questions do you think they would ask you first?
What would you need to explain in addition (what is important, but not obvious)?

Part 4: Focus Area 2 - Perceived Accuracy & Reliability (10 Minutes)

Goal: Assess RQ1 Accuracy and RQ3 Reliability, Limitations.

A. System Context Diagram (5 Minutes)

- Guiding Question:
 1. Based on your expert knowledge of the project, how accurately does this diagram represent the *actual* system context regarding external systems and users? (Stakeholders)
 2. Comparing this diagram to your mental model of the project, what are the most significant deviations regarding external systems and actors? Hallucinations or missing elements?
- Follow-up Questions:
 1. Would you rather add this diagram to the project documentation or have none instead? Why?

B. Container Diagram (5 Minutes)

- Guiding Question:
 1. Looking at the technical containers again (APIs, Databases, etc.), do these distinct units match the actual deployment reality of the project?
- Follow-up Questions:
 2. If not, do you think it would be beneficial to change the actual system to match this diagram
 3. Looking specifically at the [Container Name] container - does the description text inside the box match your mental model of that service?
 4. Do the directional relationships (the arrows) correctly represent the dependency flow between these containers?
 5. Does this diagram oversimplify any complex technical interactions that are critical for the system's success?
 6. If you encountered this diagram in the project repository today, to what extent would you rely on it for decision making? (Follow up: What specific verification would you feel compelled to do?)

Part 5: Focus Area 3 - Perceived Usefulness (10 Minutes)

Goal: Assess RQ3 Usefulness and Workflow Integration.

Scenario: Imagine this workflow runs automatically every night based on the day's Work Items, providing a steady, up-to-date architecture diagram.

A. System Context Diagram (5 Minutes)

- Guiding Question:
 1. Walk me through a typical week in your role. If this auto-generated artifact were available to you every morning, specifically where in your current process would you pull it up or use it?
- Follow-up Questions:
 1. You mentioned [specific meeting/process]. How would bringing this visualization into that conversation change the dynamic with your stakeholders?
 2. In the context of the project, how would this diagram alter the way you explain technical boundaries to non-technical peers compared to how you do it today? (Both diagrams)

3. If you saw this diagram on a slide at the very start of a new project, how might that shape your initial understanding of the solution scope?
4. In which specific meetings or process steps (e.g., Refinement, Client Kick-off) would this artifact provide the most value in your opinion?
5. What is the one critical change this diagram would need for you to consider it 'production-ready' for a client presentation?

B. Container Diagram (5 Minutes)

- Guiding Question:
 1. Regarding the more technical Container diagram: How would having this detailed technical view available on demand impact the way you **make** or document architectural decisions?
- Follow-up Questions:
 2. In scenarios like onboarding or refinement, what specific 'pain points' could this automation solve for you?
 3. How do you envision interacting with this result: Do you see it more as a validation tool to check requirements, a 'canvas' to start your design, or something else?
 4. Comparing manual diagramming to this automated approach, what do you see as the single most valuable improvement this brings to your workflow?
 5. When you weigh the benefits of 'speed of creation' versus 'consistency of documentation' which factor provides more value to you in your daily work, and why?
 6. If you could change one thing about this output to make it indispensable for your team, what would it be?

Part 6: Closing & Reflection (2 Minutes)

Goal: Catch-all for missed insights and wrap up.

Guiding Question:

1. Reflecting on our discussion about understandability, accuracy and usefulness: What is the single most critical factor that would determine whether you would trust and use this tool today?
2. Is there anything else that you would like to mention?

Thank you.